

Cours Supabase pour Lokafête

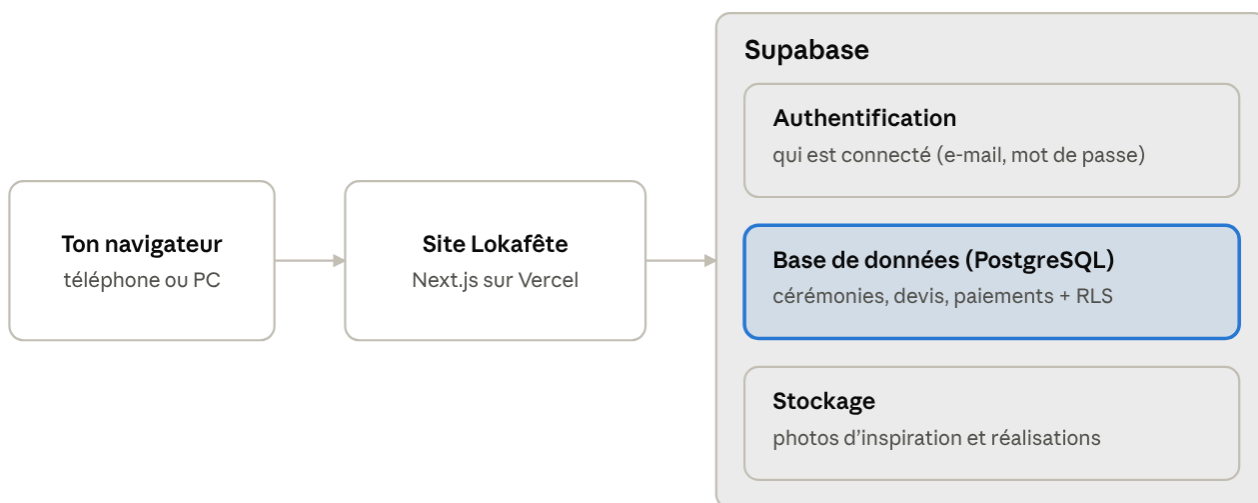
Oct 7, 2026 · @Kotchami

Un cours pas à pas pour comprendre Supabase et faire fonctionner la base de données, la connexion et les fichiers de Lokafête.

1. Comprendre Supabase

Supabase est le « coffre-fort » de Lokafête : il garde les comptes des prestataires, leurs données (cérémonies, devis, paiements) et leurs photos. Ton site, hébergé sur Vercel, lui envoie des demandes ; Supabase répond seulement si la personne a le droit.

Le site parle à Supabase, qui garde comptes, données et photos



Comment Lokafête utilise Supabase · 3 briques

Les trois briques que Lokafête utilise :

Brique	À quoi ça sert	Exemple dans Lokafête
Authentification	Créer des comptes, connecter, déconnecter	Un décorateur s'inscrit avec son e-mail
Base de données	Ranger les informations dans des tableaux	La cérémonie « Mariage famille A. », son devis, ses paiements
Stockage	Garder des fichiers	Les photos de la planche d'inspiration

La base de données est protégée par la **RLS** (Row Level Security) : chaque prestataire ne voit que ses propres lignes. C'est la règle la plus importante du cours (leçon 3).

À retenir : Supabase = comptes + données + fichiers, avec des règles de sécurité pour chacun.

2. Le tableau de bord

Tout se passe sur supabase.com/dashboard, dans ton organisation **Lokafête** et ton projet **lokafete**. Les icônes de la barre de gauche mènent aux pages dont tu auras besoin :

Page	Ce que tu y fais
Table Editor	Voir et modifier les données comme dans un tableur
SQL Editor (icône <code>>_</code>)	Exécuter du code SQL, par exemple créer toutes les tables d'un coup
Authentication	Voir les comptes inscrits, régler les e-mails et les adresses autorisées
Storage	Gérer les dossiers de fichiers (photos)
Project Settings (engrenage)	Retrouver l'adresse du projet et les clés
Connect (bouton vert en haut)	Copier rapidement l'adresse et la clé pour Next.js

Le bandeau du haut indique où tu es : organisation « Lokafête », projet « lokafete », branche « main » (PRODUCTION). Les noms exacts des menus peuvent changer légèrement avec les mises à jour de Supabase.

3. Leçon 1 : la base de données

Une base de données, c'est un ensemble de **tableaux** (appelés *tables*) reliés entre eux. Imagine le cahier d'un décorateur découpé en plusieurs carnets : un pour les clients, un pour les cérémonies, un pour les paiements.

Le vocabulaire

- **Table** : un tableau, par exemple `ceremonies`.
- **Colonne** : une information, par exemple `date`, `lieu`, `nb_invites`.
- **Ligne** : un élément, par exemple la cérémonie « Mariage famille A. ».
- **id** : le numéro unique de chaque ligne, créé automatiquement.

- **Relation** : une colonne qui pointe vers une autre table. La cérémonie a une colonne `client_id` qui dit à quel client elle appartient.

Exemple

Table clients		Table ceremonies		
id	nom	titre	date	client_id
12	Famille A.	Mariage	11/10	12
13	Famille K.	Funérailles	18/10	13

La cérémonie « Mariage » pointe vers le client n° 12 : c'est la famille A.

Deux façons de travailler

1. **SQL Editor** : pour créer ou modifier la structure (les tables). On colle un script SQL complet et on clique sur **Run**. C'est ainsi qu'on installe le schéma de Lokafête (`app/database/schema.sql`).
2. **Table Editor** : pour regarder les données, comme un tableur. Pratique pour vérifier qu'une inscription a bien créé une ligne.

Les 3 règles du SQL Editor

1. Toujours ouvrir une **nouvelle requête vide** (*New query*).
2. Coller le fichier **en entier** (Ctrl + A, Ctrl + C dans ton éditeur, puis Ctrl + V).
3. Cliquer sur **Run** sans rien sélectionner : si du texte est surligné, Supabase n'exécute que la sélection.

Résultat attendu : « Success. No rows returned », puis les tables apparaissent dans le Table Editor.

4. Leçon 2 : l'authentification

L'authentification gère les comptes. Lokafête utilise l'inscription par **e-mail et mot de passe**, avec un e-mail de confirmation.

Ce qui se passe quand un décorateur s'inscrit

1. Il remplit le formulaire d'inscription sur Lokafête.
2. Supabase crée le compte (visible dans **Authentication › Users**) et envoie un e-mail de confirmation.
3. Il clique sur le lien : Supabase le renvoie vers ton site, à l'adresse `/auth/confirm`.
4. Le site le connecte et crée son organisation (fonction `creer_organisation`).

Les réglages à connaître

Page	Réglage	Valeur pour Lokafête
Authentication › URL Configuration	Site URL	<code>https://lokafete.vercel.app</code>
Authentication › URL Configuration	Redirect URLs	<code>http://localhost:3000/auth/confirm</code> et <code>https://lokafete.vercel.app/auth/confirm</code>
Authentication › Sign In / Providers	Fournisseurs	E-mail activé
Authentication › Emails	Modèles d'e-mails	À traduire en français avec le nom Lokafête
Authentication › Emails › SMTP Settings	Service d'envoi	À configurer avant le lancement (par exemple Resend)

Pourquoi les Redirect URLs comptent : par sécurité, Supabase ne renvoie l'utilisateur que vers les adresses de cette liste. Une adresse absente, et le lien de confirmation échoue.

Limite importante : le service d'e-mail fourni par Supabase est fait pour les tests. Il n'envoie que quelques e-mails par heure et ses messages tombent souvent dans les spams. Pour de vrais clients, configure un service d'envoi (SMTP).

5. Leçon 3 : la sécurité RLS

La **RLS** (Row Level Security, « sécurité par ligne ») décide, ligne par ligne, qui peut lire ou modifier quoi. Sans elle, n'importe quel utilisateur connecté pourrait voir les cérémonies de tous les prestataires.

L'image à retenir : chaque ligne porte une étiquette `organisation_id`. Une règle dit : « tu ne vois que les lignes dont l'étiquette est celle de ton organisation ».

Si Awa (Déco A) demande...	Supabase répond
Ses cérémonies	Oui, toutes les siennes
Les cérémonies du traiteur B	Rien : aucune ligne ne s'affiche
Ajouter un client chez le traiteur B	Refusé : « violates row-level security policy »

Les réflexes

- La RLS doit être **activée sur chaque table**. Dans le Table Editor, une table sans RLS affiche un avertissement : ne le laisse jamais.
- Chaque nouvelle table a besoin de ses **règles** (*polices*). Demande toujours à Claude Code de les écrire en même temps que la table.
- Les règles se voient dans **Authentication** › **Polices** ou dans le Table Editor, sur chaque table.
- La clé `service_role` **contourne** la RLS. C'est pour ça qu'elle ne doit jamais sortir du serveur (leçon 4).

Le test simple : crée deux comptes avec deux e-mails différents. Ajoute une cérémonie avec le premier, connecte-toi avec le second : il ne doit pas la voir.

6. Leçon 4 : les clés et les variables d'environnement

Pour parler à Supabase, le site a besoin de deux informations : **l'adresse du projet** et **une clé**. On les range dans des *variables d'environnement*, jamais directement dans le code.

Les trois valeurs à connaître

Valeur	Exemple	Publique ?	Où la mettre
Adresse du projet	<code>https://xkwwvhcvnwdt1cwvlhee.supabase.co</code>	Oui	<code>NEXT_PUBLIC_SUPABASE_URL</code>
Clé anon (ou <i>publishable</i>)	commence par <code>eyJ...</code> ou <code>sb_publishable_...</code>	Oui, protégée par la RLS	<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>
Clé <code>service_role</code> (ou <i>secret</i>)	commence par <code>eyJ...</code> ou <code>sb_secret_...</code>	Non, jamais	Seulement côté serveur, plus tard (paiements)

On les trouve dans **Project Settings** › **API** (ou *Data API / API Keys*), ou avec le bouton vert **Connect**.

Le format exact de l'adresse : `https://<identifiant>.supabase.co`, sans `/` final, sans `/rest/v1`, sans guillemets. Une adresse mal écrite donne l'erreur « Invalid path specified in request URL ».

Deux endroits à remplir, toujours les deux

1. **Sur ton ordinateur** : le fichier `.env.local` à la racine du projet (bien écrit `.env.local`, avec le `v`). Il n'est jamais envoyé sur GitHub. Après une modification, redémarre `pnpm dev`.
2. **Sur Vercel** : *Environment Variables*, mêmes noms, mêmes valeurs, cochées pour Production et Preview. Après une modification, relance un déploiement (**Redeploy** sur le dernier).

Le préfixe `NEXT_PUBLIC_` veut dire « visible dans le navigateur ». C'est normal et sans danger pour l'adresse et la clé `anon`. Vercel affiche un avertissement : garde le préfixe. Ne mets jamais la clé `service_role` dans une variable `NEXT_PUBLIC_`.

7. Leçon 5 : le stockage de fichiers

Le stockage (**Storage**) garde les fichiers : photos d'inspiration envoyées par les clients, photos des réalisations pour le portfolio, logos des prestataires. La base de données ne garde que le **chemin** du fichier (colonne `chemin_stockage` de la table `photos`).

Le vocabulaire

- **Bucket** : un grand dossier, par exemple `photos`.
- **Chemin** : l'emplacement du fichier dans le bucket. Bonne pratique pour Lokafête : `<organisation_id>/<ceremonie_id>/<nom-du-fichier>.jpg`.
- **Public ou privé** : un bucket public donne une adresse visible par tous ; un bucket privé demande une autorisation.

Ce que je te conseille

Bucket	Contenu	Visibilité
<code>photos</code>	Planches d'inspiration, photos de chantier	Privé, protégé par des règles comme la RLS
<code>portfolio</code>	Photos que le prestataire choisit de montrer	Public, pour le lien partageable
<code>logos</code>	Logos sur les devis	Public

Comme pour les tables, un bucket privé a besoin de **règles** : un prestataire ne doit accéder qu'aux fichiers rangés sous son `organisation_id`.

Taille des photos : les photos de téléphone sont lourdes et les connexions parfois lentes. Demande à Claude Code de **réduire les images avant l'envoi** (par exemple 1600 pixels de large maximum). L'offre gratuite de Supabase a un quota de stockage limité.

8. Leçon 6 : travailler avec Claude Code

Claude Code écrit le code, mais il ne peut pas cliquer dans Supabase ni dans Vercel à ta place. La répartition :

Claude Code fait	Toi, tu fais
Écrire les tables, les règles RLS et le code du site	Coller les scripts SQL dans le SQL Editor et cliquer sur Run
Lire <code>.env.local</code> et vérifier son format	Copier les clés depuis Supabase et remplir Vercel
Lancer <code>pnpm build</code> et corriger les erreurs	Régler Authentication › URL Configuration
Commiter et pousser sur GitHub	Vérifier le déploiement sur Vercel

Consignes prêtes à l'emploi

Ajouter une table :

J'ai besoin d'une nouvelle table pour [usage]. Propose-moi d'abord le script SQL avec les colonnes, la clé `organisation_id`, la RLS activée et ses règles. Mets-le dans un fichier séparé `app/database/migrations/AAAA-MM-JJ-nom.sql` que je collerai dans le SQL Editor. Ensuite, mets à jour les types TypeScript.

Vérifier la configuration :

Vérifie ma configuration Supabase sans afficher les clés en entier : format de `NEXT_PUBLIC_SUPABASE_URL`, présence de la clé anon, noms des variables utilisés dans le code, et teste la connexion.

Comprendre une erreur :

Voici le message d'erreur exact : [colle-le]. Explique-moi la cause en une phrase, dis-moi si elle vient du code ou d'un réglage Supabase ou Vercel, puis corrige ce qui relève du code.

La bonne habitude : toute modification de la base passe par un **fichier de migration** dans le projet, jamais par des clics improvisés dans le Table Editor. Ainsi, le projet garde l'historique exact de ta base.

9. Dépannage

Les erreurs déjà rencontrées sur Lokafête, et comment les régler.

Ce que tu vois	Cause	Solution
<code>syntax error at or near "..."</code> , LINE 1 au milieu du fichier	Script collé en partie, ou texte sélectionné au moment du Run	Nouvelle requête vide, coller tout le fichier, Run sans sélection
<code>... already exists</code>	Une partie du script a déjà été exécutée	Lancer d'abord un script de nettoyage, puis le schéma complet
Table Editor vide (« No tables or views »)	Le schéma n'a jamais été appliqué	Coller <code>app/database/schema.sql</code> dans le SQL Editor
<code>Invalid path specified in request URL</code>	Adresse du projet mal écrite	<code>https://<identifiant>.supabase.co</code> , sans chemin ni <code>/</code> final
Variables vides en local	Fichier mal nommé (<code>.en.local</code>)	Renommer en <code>.env.local</code> , redémarrer <code>pnpm dev</code>
Déploiement Vercel en Error : « NEXT_PUBLIC_SUPABASE_URL ... doivent être définies »	Variables absentes sur Vercel	Les ajouter dans Environment Variables, puis Redeploy
Déploiement Vercel Blocked	E-mail Git non relié au compte	<code>git config --global user.email</code> avec l'e-mail vérifié de GitHub
Le lien de confirmation mène à une erreur	Adresse absente des Redirect URLs	Ajouter <code>.../auth/confirm</code> dans URL Configuration
Pas d'e-mail reçu	Spam, ou limite du service d'e-mail de test	Regarder les spams, attendre, configurer un SMTP
<code>new row violates row-level security policy</code>	La RLS refuse l'écriture	Normal si on écrit chez un autre ; sinon, vérifier <code>organisation_id</code> et les règles

Ce que tu vois	Cause	Solution
Projet en pause	Offre gratuite, une semaine sans activité	Cliquer sur Restore dans le tableau de bord

La méthode en cas de doute : copie le message d'erreur **exact** et demande à Claude Code s'il vient du code, de Supabase ou de Vercel.

10. Avant le lancement

Coche chaque point avant d'ouvrir Lokafête à de vrais clients.

- ☐ La RLS est activée sur **toutes** les tables, et le test à deux comptes est réussi.
- ☐ La clé `service_role` n'apparaît nulle part dans le code ni dans une variable `NEXT_PUBLIC_`.
- ☐ Un service d'envoi d'e-mails (SMTP, par exemple Resend) est configuré.
- ☐ Les e-mails (confirmation, mot de passe oublié) sont traduits en français, avec le nom Lokafête.
- ☐ Site URL et Redirect URLs pointent vers l'adresse définitive (`lokafete.com` le jour venu).
- ☐ Le mot de passe de la base est rangé dans un gestionnaire de mots de passe.
- ☐ La double authentification est activée sur ton compte Supabase.
- ☐ Les buckets de stockage privés ont leurs règles.
- ☐ L'offre payante est envisagée dès les premiers clients : pas de mise en pause, sauvegardes quotidiennes.
- ☐ Les traitements de données sont déclarés à l'APDP, et la politique de confidentialité mentionne l'hébergement hors du Bénin.

11. Exercices pratiques

Fais-les dans l'ordre : chacun vérifie une leçon.

- ☐ **Exercice 1, tableau de bord** : ouvre ton projet et trouve les 6 pages du tableau de la partie 2.
- ☐ **Exercice 2, base de données** : applique `app/database/schema.sql` dans le SQL Editor, puis compte les tables dans le Table Editor.
- ☐ **Exercice 3, authentification** : inscris-toi sur ton site, retrouve ton compte dans Authentication › Users, puis ta ligne dans la table des organisations.

- ☐ **Exercice 4, RLS** : crée un second compte avec un autre e-mail. Ajoute un article avec le premier ; vérifie que le second ne le voit pas.
- ☐ **Exercice 5, variables** : demande à Claude Code de vérifier ta configuration, puis compare les noms avec ceux de Vercel.
- ☐ **Exercice 6, stockage** : crée un bucket privé `photos` et téléverse une image à la main pour voir où elle se range.
- ☐ **Exercice 7, dépannage** : ouvre le dernier déploiement Vercel, trouve ses Build Logs et explique en une phrase s'il a réussi et pourquoi.

Quand les 7 cases sont cochées, tu maîtrises tout ce dont Lokafête a besoin pour la phase 1 de la feuille de route.